Full Stack Home Assignment — React

action-item.co.il | Tel: 072-2512344 | contact@action-item.co.il

Introduction

You'll build a small full-stack app: a Node + TypeScript backend and a React frontend that displays and persists random user profiles.

- Backend: Node + TypeScript
- Frontend: React (functional components + hooks)
- Time budget: ~4 hours
- Submission: Public Git repo (preferred) or zipped sources without `node_modules`

You may use any 3rd-party SDK or library. Be ready to defend your choices.

AI Policy — Read This Carefully

Using AI assistants (Claude, Copilot, Cursor, ChatGPT, etc.) is explicitly permitted and expected. This reflects how you would actually work on our team.

However, we evaluate your judgment, not your typing speed. When everyone has the same tools, the difference between candidates shows up in the choices they made, the tradeoffs they understood, and the things they noticed. We can't see any of that from the code alone, AI-generated code looks broadly similar regardless of who pasted the prompt.

That's why we ask for two short text documents alongside the code: `AI_USAGE.md` and `DECISIONS.md` (details below).

Two rules:

1. Disclose what you used in `AI_USAGE.md`. A short paragraph is fine.
2. You are responsible for every line. "The AI wrote it that way" is not an acceptable answer to any question about your code. If you can't defend a line, delete it or rewrite it until you can.

What We're Evaluating

In rough order of weight:

1. Judgment — the choices you made and how you defend them in `DECISIONS.md`
2. Code quality — clarity, structure, error handling, types
3. Working software — it runs, the flows work end-to-end
4. Communication — README, AI_USAGE.md, DECISIONS.md
5. React/Node fluency — idiomatic use of the frameworks

A polished half-built app beats a feature-complete mess. We mean this.

Front Task

A React app that displays random people and lets the user persist a subset of them.

- Use https://randomuser.me/ to fetch a list of 10 people.
- Use a state management approach of your choice (Redux, Zustand, Context, TanStack Query, etc.). Be ready to defend it.

Screen 0 — Home

Two buttons:

- Fetch → opens Screen 1 (random user API)
- History → opens Screen 2 (saved users from your backend)

Screen 1 — Random List

List view. Each row shows:

- Thumbnail
- Name (title + first + last)
- Gender
- Country
- Phone
- Email

Plus:

- Filter by name and country. Your call how: one input, two inputs, debounced, instant. Document the choice.
- Clicking a row opens Screen 3.

Screen 2 — Saved Profiles

Identical to Screen 1, but the data comes from your backend.

Screen 3 — Profile Detail

Opened from Screen 1 or Screen 2.

- Large image
- Gender
- Editable Name field
- Age + year of birth
- Address: street number + name, city, state
- Contact: email, phone
- Four buttons:
  - Save — only when the profile came from Screen 1 (not yet in the DB). Persists to backend.
  - Delete — only when the profile came from Screen 2 (already in the DB). Removes from backend.
  - Update — name field is editable.
    - If the profile is saved, update the backend.
    - If the profile is not saved, update the in-memory list on Screen 1.
  - Back — navigates back.

Bidirectional text requirement. Screen 3 must support mixed Hebrew/English content correctly. The layout should be RTL (the labels "Gender", "Name", "Address", etc. should be rendered in Hebrew), but LTR data inside that layout: email, phone, street number, the editable Latin name field must remain visually LTR and behave correctly when editing. You decide how to handle direction on the `<input>` elements, how the form aligns, and what to do with the buttons. We are looking for awareness of BiDi, not pixel-perfect typography. Document your approach in `DECISIONS.md`.

Back Task

- Node + TypeScript
- Any persistence layer (SQLite, Postgres, JSON file, in-memory, your call, defend it)
- Minimum viable API surface. We are not looking for full REST maturity, auth, or RBAC.
- No authentication required.

A thin solution is fine. A sloppy solution is not. Even with only 3–4 endpoints we want to see proper software design.

Required Deliverables

You must submit all of the following.

1. Working code

Client and server clearly separated (two folders or two repos). Each with a `README.md` containing:

- Prerequisites (Node version, etc.)
- Install + run instructions
- Any environment variables

2. `DECISIONS.md` — a plain text/Markdown file, max 1 page

Write up 3 specific decisions you made and the tradeoff for each. Pick ones that were actually interesting, not "I chose Vue 3 because the spec said so."

Examples of the kind of thing we mean:

- "I put the filter in the store rather than as local component state because..."
- "I chose SQLite over a JSON file because... The tradeoff in production would be..."
- "I debounced the filter input at 200ms because..."

Also include in this file:

- Your approach to the RTL/LTR mixed-direction requirement on Screen 3
- Corners you cut deliberately and what you'd do in production
- Your extension (see below). What you added, why, and what you'd do next

3. `AI_USAGE.md` — a short plain text/Markdown file

List the AI tools you used and roughly what for. One paragraph is fine. Honest disclosure only, we're using this to read the rest of the submission in context, not to score you on it.

4. One extension of your choice (~30 min of the budget)

After implementing the spec, add one thing we didn't ask for that you think would make this better. Optimistic updates, retry on failure, a loading skeleton, an accessibility pass, keyboard nav, error boundaries, tests for one critical path, your call.

Cover it in `DECISIONS.md` in 100 words or fewer: what you added, why you picked it over other options, and what you'd build next if you had another hour.

This is the single most important part of the submission. It tells us what you think "good" looks like.

Guidelines

1. Only a working application will be considered for review.
2. Read the spec twice. There are implicit requirements.
3. No UI design is provided on purpose. Use a component library or plain CSS. Defend the choice.
4. Cut corners deliberately. Note them in `DECISIONS.md` with what you'd do in production.
5. We prefer a working, well-designed solution with less functionality over a sprawling one with bugs.

Submission

- Public Git repo with clear Client/Server separation, or a zip without `node_modules`
- README at the top level with project overview and how to run both sides
- Deployment to a public cloud (Vercel, Render, Fly, AWS, etc.) is a plus, not a requirement

Good luck.